L.Aravind Reddy

Batch-38

Roll-no- 2303A52028

Task Description -1 (Refactoring – Eliminating Repetitive Logic) • Task: Use AI-assisted coding techniques to restructure a Python program that performs repeated calculations, transforming it into a reusable and maintainable design. Instructions: • Ask AI to locate repeated computations within the program. • Encapsulate repeated logic into reusable functions. • Verify that program output remains unchanged after refactoring. • Add proper docstrings to all defined functions. Sample Code: length = 8 width = 4 print("Area:", length * width) print("Perimeter:", 2 * (length + width)) length = 12 width = 6 print("Area:", length * width) print("Perimeter:", 2 * (length + width)) Expected Output: A refactored Python script that uses a single reusable function to compute area and perimeter without repeating code.

```python
from typing import Tuple

def compute_area_perimeter(length: float, width: float) -> Tuple[float, float
    area = length * width
    perimeter = 2 * (length + width)
    return area, perimeter


def main() -> None:
    length = 8
    width = 4
    area, perimeter = compute_area_perimeter(length, width)
    print("Area:", area)
    print("Perimeter:", perimeter)

    length = 12
    width = 6
    area, perimeter = compute_area_perimeter(length, width)
    print("Area:", area)
    print("Perimeter:", perimeter)


if __name__ == "__main__":
    main()
```

(variable) perimeter: float

```
Area: 32
Perimeter: 24
Area: 72
Perimeter: 36
```

Task Description -2 (Refactoring – Modularizing Inline Calculations) • Task: Use AI to refactor a Python script by extracting repeated inline calculations into reusable and well-documented functions. Instructions: • Identify repeated calculation patterns in the code. • Convert them into reusable functions. • Ensure readability and proper documentation. Sample Code: amount = 1000 discount = amount * 0.10 final_amount = amount - discount print("Final Amount:", final_amount) amount = 2000 discount = amount * 0.10 final_amount = amount - discount print("Final Amount:", final_amount) Expected Output: A modular Python program that uses a reusable function to compute final amounts for different inputs.

```python
from typing import Union


def compute_final_amount(amount: Union[int, float], rate: float = 0.10) -> f
    discount = amount * rate
    final_amount = amount - discount
    return final_amount


def main() -> None:
    amount = 1000
    final_amount = compute_final_amount(amount, 0.10)
    print("Final Amount:", final_amount)

    amount = 2000
    final_amount = compute_final_amount(amount, 0.10)
    print("Final Amount:", final_amount)


if __name__ == "__main__":
    main()
```

```
Final Amount: 900.0
Final Amount: 1800.0
```

Task Description -3 (Refactoring – Improving Loop and Conditional Performance) • Task: Apply AI-based suggestions to enhance the performance of Python code that relies on nested loops and repeated conditional checks. Instructions: • Use AI to recommend more efficient data structures or logic simplifications. • Replace inefficient constructs while preserving program behavior. • Compare execution efficiency before and after refactoring. Sample Code: students = ["Anil", "Bhavya", "Charan", "Divya"] check_list = ["Charan", "Esha", "Bhavya"] for name in check_list: exists = False for student in students: if name == student: exists = True if exists: print(name, "found") else: print(name, "not found")

Expected Output: An optimized Python program that performs faster lookups using efficient logic, along with execution performance comparison.

```python
from typing import Iterable, Dict


def nested_search(check_list: Iterable[str], students: Iterable[str]) -> Dic
    results = {}
    for name in check_list:
        exists = False
        for student in students:
            if name == student:
                exists = True
                break
        results[name] = exists
    return results


def set_search(check_list: Iterable[str], students: Iterable[str]) -> Dict[s
    student_set = set(students)
    return {name: (name in student_set) for name in check_list}


def print_results(results: Dict[str, bool]) -> None:
    for name, found in results.items():
        if found:
            print(name, "found")
        else:
            print(name, "not found")


def benchmark(students, check_list, repeats: int = 1000) -> None:
    import time

    # Warm-up (avoid cold-start bias)
    nested_search(check_list, students)
    set_search(check_list, students)

    start = time.perf_counter()
    for _ in range(repeats):
```

```python
def benchmark(students, check_list, repeats: int = 1000) -> None:
    t_nested = time.perf_counter() - start

    start = time.perf_counter()
    for _ in range(repeats):
        set_search(check_list, students)
    t_set = time.perf_counter() - start

    print(f"Nested search: {t_nested:.6f} s")
    print(f"Set search:    {t_set:.6f} s")
    if t_set > 0:
        print(f"Speedup: {t_nested / t_set:.2f}x faster (nested / set)")


def main() -> None:
    students = ["Anil", "Bhavya", "Charan", "Divya"]
    check_list = ["Charan", "Esha", "Bhavya"]

    print("-- nested_search (sample) --")
    print_results(nested_search(check_list, students))

    print("\n-- set_search (sample) --")
    print_results(set_search(check_list, students))

    large_students = [f"Student{i}" for i in range(2000)] + students

    large_check = [f"Student{i}" for i in range(0, 2000, 250)] + ["Esha", "Bh

    print("\n-- Benchmark (larger dataset) --")
    benchmark(large_students, large_check, repeats=300)


if __name__ == "__main__":
    main()
```

```
s/3.10.14/bin/python /Users/lakkiaravindreddy/assiangments/lookup_opt.py
Esha not found
Bhavya found

-- set_search (sample) --
Charan found
Esha not found
Bhavya found

-- Benchmark (larger dataset) --
Nested search: 0.106916 s
Set search:    0.015309 s
Speedup: 6.98x faster (nested / set)
```

Task Description -4 (Refactoring – Replacing Hardcoded Values with Constants) • Task: Utilize AI assistance to improve maintainability by replacing hardcoded numerical values with named constants. Instructions: • Detect hardcoded numeric values used directly in expressions. • Define descriptive constants at the beginning of the program. • Update calculations to use these constants without changing results. Sample Code: print("Simple Interest:", (1000 * 2 * 5) / 100) print("Simple Interest:", (2000 * 2 * 5) / 100) Expected Output: A revised Python program using constants such as RATE and TIME for clearer and maintainable calculations..

```python
from typing import Union
RATE = 2.0          # interest rate (percent)
TIME = 5.0          # time period (years)
PERCENT = 100.0     # divisor to convert percentage to fraction
PRINCIPAL_A = 1000.0
PRINCIPAL_B = 2000.0


def compute_simple_interest(principal: Union[int, float], rate: float = RATE,
    interest = (principal * rate * time) / percent
    return interest


def add_numbers(first: Union[int, float], second: Union[int, float]) -> Unio
    return first + second


def main() -> None:
    si1 = compute_simple_interest(PRINCIPAL_A)
    print("Simple Interest:", si1)

    si2 = compute_simple_interest(PRINCIPAL_B)
    print("Simple Interest:", si2)
    first_value = 50
    second_value = 30
    total = add_numbers(first_value, second_value)
    print(total)


if __name__ == "__main__":
    main()
```

```
lakkiaravindreddy@lakkis-MacBook-Air ~ % /Users/lakkiaravindreddy/.pyenv/version
s/3.10.14/bin/python /Users/lakkiaravindreddy/assiangments/simple_interest.py
Simple Interest: 100.0
Simple Interest: 200.0
80
lakkiaravindreddy@lakkis-MacBook-Air ~ %
```

Task Description -5 (Re factoring – Enhancing Variable Naming and Code Clarity) • Task: Apply AI tools to improve code readability by renaming unclear variables and adding meaningful comments.. Instructions: • Replace non-descriptive variable names with

meaningful identifiers. • Add inline comments where the logic may be unclear. • Ensure program output remains the same. Sample Code: x = 50 y = 30 z = x + y print(z) Expected Output: A Python program with clear variable names and explanatory comments that improve readability without altering functionality.

```
lakkiaravindreddy@lakkis-MacBook-Air ~ % /Users/lakkiaravindreddy/.pyenv/versions/3.10.14/bin/python /Users/lakkiaravindreddy/assiangments/variable.py
80
```

```python
def add_two_numbers(first_number: int, second_number: int) -> int:
    sum_result = first_number + second_number
    return sum_result


def main() -> None:
    first_number = 50
    second_number = 30
    total = add_two_numbers(first_number, second_number)
    print(total)
if __name__ == "__main__":
    main()
```